

F1 PIT WALL AI

SYSTEM ARCHITECTURE & DATABASE SEEDING

Module 1: Production Relational Agent Pipeline

This guide is an end-to-end implementation manual for building an AI-powered Formula 1 pit wall telemetry assistant using NestJS, LangChain, SQLite, and Angular 19.

1. System Architecture Overview

- Backend: NestJS POST SSE stream endpoint emitting real-time agent tool execution status and token deltas.
- AI Layer: LangChain ReAct Agent with GPT-4o-mini utilizing dynamic SQLite introspection & query tools.
- Frontend: Angular 19 Standalone Component consuming ReadableStream with AsyncGenerators and Signals.

2. Step 1: Database Setup & Seeding

Initialize your backend project and install core dependencies:

```
npm init -y
npm install @nestjs/common @nestjs/core @nestjs/platform-express rxjs reflect-metadata better-sqlite3 dotenv
npm install -D @types/better-sqlite3 @types/node tsx typescript
```

Database Seeding Script (src/seed.ts)

Create and seed the SQLite relational store with drivers, races, and pit stop tables:

```
import Database from 'better-sqlite3';
const db = new Database('f1_pitwall.db');
db.pragma('foreign_keys = ON');

db.exec(`
  CREATE TABLE IF NOT EXISTS drivers (driver_id TEXT PRIMARY KEY, name TEXT, nationality TEXT);
  CREATE TABLE IF NOT EXISTS races (race_id TEXT PRIMARY KEY, year INT, name TEXT, circuit_name TEXT);
  CREATE TABLE IF NOT EXISTS results (result_id INT PRIMARY KEY, race_id TEXT, driver_id TEXT, position INT, points REAL);
  CREATE TABLE IF NOT EXISTS pit_stops (stop_id INT PRIMARY KEY, race_id TEXT, driver_id TEXT, lap INT, duration TEXT);
`);

// Seed drivers & races
db.prepare('INSERT OR IGNORE INTO drivers VALUES (?, ?, ?)').run('verstappen', 'Max Verstappen', 'Dutch');
db.prepare('INSERT OR IGNORE INTO races VALUES (?, ?, ?, ?)').run('2021_monaco', 2021, 'Monaco GP', 'Monaco');
db.prepare('INSERT OR IGNORE INTO results VALUES (1, "2021_monaco", "verstappen", 1, 25)').run();
console.log('f1_pitwall.db successfully created and seeded.');
db.close();
```

F1 PIT WALL AI

LANGCHAIN TOOLS & REACT AGENT SERVICE

Step 2: Dynamic Schema & SQL Function Calling

3. Step 2: Database Tools & Agent Service

Install the LangChain ecosystem dependencies:

```
npm install langchain @langchain/core @langchain/openai @langchain/langgraph zod
```

Custom SQLite Dynamic Tools (src/tools.ts)

```
import { DynamicStructuredTool } from '@langchain/core/tools';
import { z } from 'zod';
import Database from 'better-sqlite3';

const db = new Database('f1_pitwall.db', { readonly: true });

export const getDbSchemaTool = new DynamicStructuredTool({
  name: 'get_db_schema',
  description: 'Returns table DDL schemas. Use first to inspect tables and columns.',
  schema: z.object({}),
  func: async () => {
    const rows = db.prepare("SELECT sql FROM sqlite_master WHERE type='table'").all();
    return rows.map(r => r.sql).join('\n\n');
  }
});

export const executeSqlQueryTool = new DynamicStructuredTool({
  name: 'execute_sql_query',
  description: 'Executes a read-only SQLite SELECT statement and returns JSON rows.',
  schema: z.object({ query: z.string() }),
  func: async ({ query }) => {
    try {
      return JSON.stringify(db.prepare(query).all());
    } catch (e) {
      return JSON.stringify({ error: e.message });
    }
  }
});
```

NestJS Agent Service (src/agent.service.ts)

```

import { Injectable } from '@nestjs/common';
import { Observable } from 'rxjs';
import { ChatOpenAI } from '@langchain/openai';
import { createReactAgent } from '@langchain/langgraph/prebuilt';
import { HumanMessage, SystemMessage } from '@langchain/core/messages';
import { getDbSchemaTool, executeSqlQueryTool } from './tools.js';

@Injectable()
export class AgentService {
  private agent = createReactAgent({
    llm: new ChatOpenAI({ modelName: 'gpt-4o-mini', temperature: 0 }),
    tools: [getDbSchemaTool, executeSqlQueryTool],
    messageModifier: new SystemMessage('You are F1 Pit Wall AI. Inspect schema and execute SQL to answer concisely.')
  });

  streamAgentEvents(prompt: string): Observable<any> {
    return new Observable((sub) => {
      (async () => {
        const stream = this.agent.streamEvents({ messages: [new HumanMessage(prompt)] }, { version: 'v2' });
        for await (const event of stream) {
          if (event.event === 'on_tool_start') sub.next({ type: 'status', data: `ðŸ”Ž Executing: ${event.name}...` });
          if (event.event === 'on_chat_model_stream' && event.data?.chunk?.content) {
            sub.next({ type: 'token', data: event.data.chunk.content });
          }
        }
        sub.next({ type: 'done', data: '[DONE]' });
        sub.complete();
      })();
    });
  }
}

```

F1 PIT WALL AI

NESTJS CONTROLLER & ANGULAR SERVICE

Step 3 & 4: SSE Controller and Fetch Stream Reader

4. Step 3: SSE Streaming Controller (src/chat.controller.ts)

```
import { Controller, Post, Body, Sse, MessageEvent } from '@nestjs/common';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';
import { AgentService } from '../agent.service.js';

@Controller('chat')
export class ChatController {
  constructor(private readonly agentService: AgentService) {}

  @Post('stream')
  @Sse()
  streamChat(@Body() body: { message: string }): Observable<MessageEvent> {
    return this.agentService.streamAgentEvents(body.message).pipe(
      map(evt => ({ data: JSON.stringify(evt) })),
    );
  }
}
```

5. Step 4: Angular 19 Streaming Client

Create an Angular 19 client application (with CSR enabled):

```
npx @angular/cli@19 new frontend --style=scss --routing=false --ssr=false
```

Chat Streaming Service (src/app/services/chat.service.ts)

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class ChatService {
  private readonly URL = 'http://localhost:3000/chat/stream';

  async *streamQuery(message: string): AsyncGenerator<{ type: string; data: string }> {
    const res = await fetch(this.URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message })
    });
    const reader = res.body!.getReader();
    const decoder = new TextDecoder('utf-8');
    let buffer = '';

    while (true) {
      const { value, done } = await reader.read();
      if (done) break;
      buffer += decoder.decode(value, { stream: true });
      const lines = buffer.split('\n');
      buffer = lines.pop() || '';

      for (const line of lines) {
        if (line.trim().startsWith('data:')) {
          yield JSON.parse(line.trim().replace(/^data:\s*/, ''));
        }
      }
    }
  }
}
```

F1 PIT WALL AI

ANGULAR SIGNALS UI & EXECUTION VERIFICATION

Step 4: Real-time UI & End-to-End Testing

6. Angular 19 Signal Component (src/app/app.component.ts)

```

import { Component, signal, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { ChatService } from '../services/chat.service';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.scss']
})
export class AppComponent {
  private chatService = inject(ChatService);
  userPrompt = signal('');
  messages = signal<Array<{ sender: string; content: string; status?: string }>>([]);
  isGenerating = signal(false);

  async sendMessage() {
    const prompt = this.userPrompt().trim();
    if (!prompt || this.isGenerating()) return;
    this.userPrompt.set('');
    this.isGenerating.set(true);

    const agentIdx = this.messages().length + 1;
    this.messages.update(m => [...m, { sender: 'user', content: prompt }, { sender: 'agent',
content: '', status: 'Thinking...' }]);

    for await (const evt of this.chatService.streamQuery(prompt)) {
      if (evt.type === 'status') {
        this.messages.update(m => m.map((msg, i) => i === agentIdx ? { ...msg, status:
evt.data } : msg));
      } else if (evt.type === 'token') {
        this.messages.update(m => m.map((msg, i) => i === agentIdx ? { ...msg, content:
msg.content + evt.data } : msg));
      } else if (evt.type === 'done') {
        this.messages.update(m => m.map((msg, i) => i === agentIdx ? { ...msg, status:
undefined } : msg));
      }
    }
    this.isGenerating.set(false);
  }
}

```

7. Complete Verification & Execution Steps

1. Seed the Database: `npx tsx src/seed.ts`
2. Launch NestJS Backend: `OPENAI_API_KEY="sk-..." npx tsx src/main.ts` (Port 3000)
3. Launch Angular Client: `npm start` (Port 4200)
4. Open `http://localhost:4200` and query: "Who won the race in Monaco in 2021?"